

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1-ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1221	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	K. NISHALINI	Reg.No.:	192521301
Department:	B.Tech IT	Date:	30/7/26

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

ASSESSMENT TOOL - 1

ANALYTICAL PROBLEM SOLVING

Q1) A Processor Performs arithmetic operations - 8-bit integers 2's Compl....

The processor stores signed integers using 8-bit 2's complement representation.

Given, +45, -23.

Required, $\rightarrow$ convert decimal numbers to 8-bit binary.

- $\rightarrow$ Perform addition and subtraction.
- $\rightarrow$ Show intermediate steps.
- $\rightarrow$ check overflow
- $\rightarrow$ Interpret the final result.

Method used:

8 bit 2's complement arithmetic

Binary addition.

Overflow detection using sign bits.

Reason: Modern processors use 2's complement because it allows addition and subtraction using the same hardware.

Step 1: Convert +45 into binary

decimal = 45

$$45 = 32 + 8 + 4 + 1$$

Binary: 00101101

Decimal	Binary
+45	00101101

Step 2: Convert 23 into 2's Complement

a) Write +23 = 00010111

b) Find 1's Complement

Invert all bits 11101000

c) Add 1

$$\begin{array}{r} 11101000 \\ + 1 \\ \hline 11101001 \end{array}$$

Therefore

Decimal	Binary
-23	11101001

Addition

$$(+45 + (-23))$$

$$\begin{array}{r} 00101101 \\ 11101001 \\ \hline 100010110 \end{array}$$

Ignore the carry

Final Answer : 00010110

Convert to decimal

$$16 + 4 + 2 = 22$$

$$\text{Result: } +45 + (-23) = +22$$

Overflow check:

It occurs when
Positive + Positive = Negative
Negative + Negative = Positive

Here positive + Negative

$\therefore$ No Overflow.

Subtraction (+45 - (-23))

$$45 - (-23) = 45 + 23$$

$$\begin{array}{r} \text{Binary } 00101101 \\ 00010111 \\ \hline 01000100 \end{array}$$

Binary 01000100

$$\text{decimal: } 64 + 4 = 68$$

$$\text{Result: } +45 - (-23) = 68$$

overflow check

Largest +ve 8bit

$$127 \text{ Result } 68$$

$$68 < 127$$

$\therefore$ No Overflow.

Accuracy of Calculation:

<u>Operation</u>	<u>Binary Result</u>	<u>Decimal Result</u>	<u>Overflow</u>
$45 + (-23)$	00010110	22	No
$45 - (-23)$	01000100	68	No

Carry look ahead adders calculate all carries simultaneously using logic equations, greatly reducing delay.

Example:

$$A = 1011 \quad B = 0110$$

Initial carry $C_0 = 0$

STEP 1:
Write bits

Bit	A	B
3	1	0
2	0	1
1	1	1
0	1	0

STEP 2:
Find Generate

Bit	A	B	$G = A \cdot B$
0	1	0	0
1	1	1	1
2	0	1	0
3	1	0	0

Therefore $G = 0010$

Find Propagate (P) STEP 3:

$$P = A \oplus B$$

Bit	A	B	P
0	1	0	1
1	1	1	0
2	0	1	1
3	1	0	1

Therefore

$$P = 1101$$

STEP 4:

Calculate Carries:

$$\begin{aligned}C_1 &= G_0 + P_0 C_0 \\&= 0 + (1 \times 0) \\&= 0\end{aligned}$$

$$\begin{aligned}C_2 &= G_1 + P_1 C_1 \\&= 1 + (0 \times 0) \\&= 1\end{aligned}$$

$$\begin{aligned}C_3 &= G_2 + P_2 C_2 \\&= 0 + (1 \times 1) \\&= 1\end{aligned}$$

$$\begin{aligned}C_4 &= G_3 + P_3 C_3 \\&= 1\end{aligned}$$

Therefore,

Carry	Value
C_0	0
C_1	0
C_2	1
C_3	1
C_4	1

STEP 5: Calculate Sum:

$$S = p \oplus \text{Carry}$$

$$S_0 = 1 \oplus 0 = 1$$

$$S_1 = 0$$

$$S_2 = 0$$

$$S_3 = 0$$

final sum:

0001

Carry out:
1

Therefore $1011 + 0110 = 10001_2$

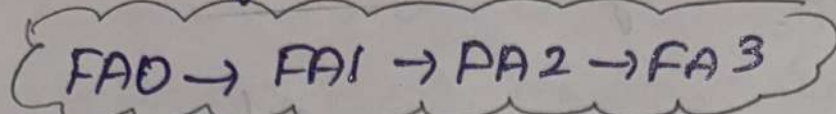
decimal check,

$$11 + 6 = 17$$

Answer: 10001_2

Time delay Comparison,

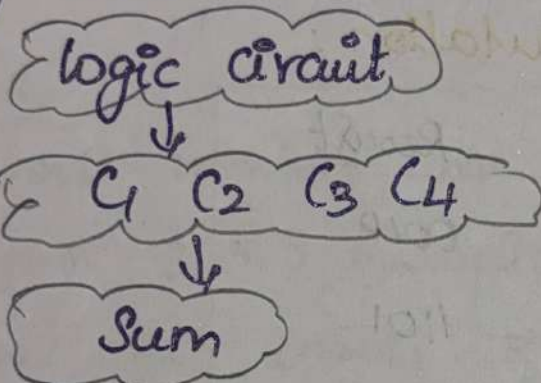
Ripple Carry Adder (RCA)



If one stage takes T

Total delay $4T$

Carry Look Ahead Adder (CLA)



Approximate delay = $2T$

Comparison Table:

Feature	Ripple Carry Adder	Carry Look-Ahead Adder
Carry generation	Sequential	parallel
Speed	Slow	very fast
Delay	High ($\approx 4T$)	Low ($\approx 2T$)
Hardware	Simple	More Complex
Processor use	Small circuits	Modern CPUs

Why CLA is faster:-

- * Computers carry all bits at the same time.
- * Eliminates ripple carry delay
- * Reduces execution time.
- * Increases processor performance.

Accuracy of Calculation:

Parameter	Result
Generate (G)	0010
Propagate (P)	1101
Carries	0, 0, 1, 1, 1
Final Sum	10001 ₂
Decimal Answer	17

Interpretation of Results:

- * Generate and propagate signals allow the processor to compute carries in advance
- * Compared with the RCA, the CLA significantly reduces computation delay
- * This improves speed makes the CLA widely used in ALUs, modern processor.

Q3) In a signal multiplication operation
a processor uses Booth's Algorithm.....

The processor must multiply two signed binary numbers using Booth's Algorithm.

Given, Multiplicand (M) = -6

Multiplier (Q) = +5.

Required,

- Convert numbers into binary
- perform Booth's multiplication step by step.
- Show the contents of the accumulator (A) Multiplier (Q) and Booth bit (Q_{-1}) after each cycle.

Methods used:

Booth's Multiplication Algorithm.

Why Booth's Algorithm:

- * It is designed for signed binary multiplication using 2's complement numbers.
- * It is widely used in processors because it is faster and more efficient than ordinary binary multiplication

STEP 1: convert numbers to 4-bit binary:

Multiplicand (-6) $+6 = 0110$

1's complement $= 1001$

Add 1 $= 1010$

Therefore $M = 1010$

Multiplier (+5)

$5 = 0101$

$\therefore Q = 0101$

Initial Values

Booth's Rules

Register

Value

$Q_0 Q_{-1}$

Operation

A

0000

00

No Operation

Q

0101

11

No operation

M

1010

01

$A = A + M$

Q_{-1}

0

10

$A = A - M$

After every operation, Perform Arithmetic Right Shift on A, Q and Q_{-1} .

Iteration 1:

Current:

$A = 0000$

$Q = 0101$

Pattern = 10

$Q_0 = 0$

$Q_{-1} = 0$

Rule :

$$A = A - M$$

$$A = 0000 - 1010 = 0110$$

Now perform Arithmetic Right Shift

Before shift :

$$A = 0110$$

$$Q = 0101$$

$$Q_{-1} = 0$$

After shift :

$$A = 0011$$

$$Q = 0010$$

$$Q_{-1} = 1$$

Iteration 2

$$Q_0 = 0$$

$$Q_{-1} = 1$$

$$\text{Pattern} = 01$$

Rule :

$$A = A + M$$

$$0011$$

$$+ 1010$$

$$1101$$

Arithmetic Right Shift

After Shift

$$A = 1110$$

$$Q = 1001$$

$$Q_{-1} = 0$$

Iteration 3

$$Q_0 = 1$$

$$Q_{-1} = 0$$

$$\text{Pattern} = 10$$

Rule :

$$A = A - M$$

$$1110$$

$$- 1010$$

$$= 0100$$

Arithmetic Right Shift

After Shift

$$A = 0010$$

$$Q = 0100$$

$$Q_{-1} = 1$$

Iteration 4

$$Q_0 = 0$$

$$\text{Pattern} = 01$$

$$Q_{-1} = 1$$

Rule:

$$A = A + M$$

$$0010$$

$$+ 1010$$

$$1100$$

Arithmetic Right Shift

After Shift

$$A = 1110$$

$$Q = 0010$$

$$Q_{-1} = 0$$

Final Product ::

Combine A and Q

$$A = 1110$$

$$Q = 0010$$

Final Answer: 11100010₂

This is the 8-bit 2's Complement representation of -30

Decimal Verification:

$$(-6) \times (+5) = -30$$

Role of Registers:

→ Accumulator (A)

→ Multiplier (R)

→ Booth Bit (R-1)

Arithmetic Shift Right:

Before Shift - 1110

After Shift - 1111

The MSB remains 1,
Preserving negative sign.

Accuracy of Calculation:

Parameter	Result
Multiplicand (M)	1010 (-6)
Multiplier (R)	0101 (+5)
Final Binary product	11100010 ₂
Decimal product	-30

Interpretation of Results:

* Booth's Algorithm correctly multiplies -6 and +5 to produce -30.

* Booth's Algorithm is preferred in modern processors because it performs signed multiplication efficiently with fewer arithmetic operations, hardware complexity.

Q.4) A Processor is required to perform division of two binary numbers.....

The Solution should:

- * Convert the no into binary
- * Show the intermediate remainder after each step.
- * Form the quotient.
- * Compare both methods.
- * Explain why non-restoring division is preferred.

Methods used:

- Restoring Division Algorithm.
- Non-Restoring Division Algorithm.

Solution Given:

STEP 1: Convert Decimal to Binary:

Dividend = 13

$13_{10} = 1101_2$

Divisor = 3

$3_{10} = 0011_2$

Registers:

* Dividend (Q) = 1101

* Divisor (M) = 0011

* Accumulator (A) = 0000

Restoring Division Algorithm:

Initial Values

A = 0000

Q = 1101

M = 0011

Iteration 1:

Shift left (A, Q)

$$A = 0001$$

$$Q = 1010$$

Subtract divisor

$$\begin{array}{r} 0001 \\ -0011 \\ \hline 1110 \end{array}$$

Since A is negative, store it

$$\begin{array}{r} 1110 \\ +0011 \\ \hline 0001 \end{array}$$

Quotient bit = 0

Iteration 2:

Shift left

$$A = 0011$$

$$Q = 0100$$

Subtract

$$\begin{array}{r} 0011 \\ -0011 \\ \hline 0000 \end{array}$$

Positive remainder:

Quotient bit = 1

Iteration 3:

Shift left

$$A = 0000$$

$$Q = 1001$$

Subtract

$$\begin{array}{r} 0000 \\ -0011 \\ \hline 1101 \end{array}$$

Negative

Restore

$$\begin{array}{r} 1101 \\ +0011 \\ \hline 0000 \end{array}$$

Quotient bit = 0

Iteration 4:

Shift left

$$A = 0001$$

$$Q = 0010$$

Subtract

$$A = 0001$$

$$Q = -0011$$

$$\begin{array}{r} 1110 \\ \hline \end{array} \text{ Negative}$$

Restore

$$\begin{array}{r} 1110 \\ +0011 \\ \hline 0001 \end{array}$$

Quotient bit = 0

Final Result

$$\text{Quotient} = 0100_2 = 4$$

$$\text{Remainder} = 0001_2 = 1$$

$$\therefore 13 \div 3 = 4 \text{ remainder } 1$$

Non Restoring Division Algorithm:

Initial Values

$$A = 0000$$

$$Q = 1101$$

$$M = 0011$$

Iteration 1:

Shift left

$$A = 0001$$

$$Q = 1010$$

Subtract M

$$\begin{array}{r} 0001 \\ -0011 \\ \hline 1110 \end{array}$$

Negative remainder

Quotient bit = 0

Iteration 2:

Since Remainder is (-)
add M instead of
restoring

$$\begin{array}{r} 1110 \\ + 0011 \\ \hline 0001 \end{array}$$

Shift left

$$A = 0011$$

$$Q = 0100$$

Positive Remainder

Quotient bit = 1

Final Result:-

$$\text{Quotient} = 0100_2 = 4$$

$$\text{Remainder} = 0001_2 = 1$$

$$\therefore 13 \div 3 = 4 \text{ remainder } 1.$$

Accuracy of Calculation:-

Method	Quotient	Remainder
Restoring division	$0100_2 (4)$	$0001_2 (1)$
Non-Restoring	$0100_2 (4)$	$0001_2 (1)$

Iteration 3:

Subtract M

$$\begin{array}{r} 0011 \\ - 0011 \\ \hline 0000 \end{array}$$

Quotient bit = 1

Iteration 4:

Shift left

$$A = 0000$$

$$Q = 1000$$

Subtract

$$\begin{array}{r} 0000 \\ - 0011 \\ \hline 1101 \end{array}$$

Negative remainder
Quotient bit = 0

Final remainder = 0001

Interpretation of Results:

* Both restoring and Non-restoring algorithm produce the same correct result: $13 \div 3 = 4$ remainder 1.

* Restoring - Extra restoration steps whenever the remainder becomes negative.

* Non Restoring - avoids unnecessary restoration, reducing execution time and improving Efficiency.

* $\therefore$ Non Restoring is preferred in modern Computer systems because it provides faster performance with fewer arithmetic operations.

Q.5) A Scientific Application requires precise handling of real numbers using the IEEE 754 standard.

The Problem requires:

- 13.25 using IEEE 754 floating Point
- Single precision (32 bit)
- Double Precision (64 bit)
- Discuss normalization, rounding and Precision loss.

Method used:

Use the IEEE 754 floating Point Standard.

Solution Process:

STEP 1: Convert -13.25 to binary

Integer Part $13_{10} = 1101_2$

Fractional Part $0.25 \times 2 = 0.5 \rightarrow 0$

$0.5 \times 2 = 1.0 \rightarrow 1$

$\therefore 0.25 = 0.01_2$

Hence $13.25 = 1101.01_2$

Since the number is negative,

$-13.25 = -1101.01_2$

STEP 2: Normalize the binary number

Move the binary point after the first 1.

$1101.01_2 = 1.10101 \times 2^3$

Single precision. (32 bits)

IEEE 754 Single Precision consist of:

Field	Bits
Sign	1
Exponent	8
Mantissa (fraction)	23

a) Sign bit = Negative number
 $\text{Sign} = 1$

b) Exponent

Actual exponent = 3

Bias = 127

Stored exponent $3 + 127 = 130$

130 in binary

10000010

c) Mantissa

From 1.10101

Removes the leading 1.

101010000000000000000000 (23 bits)

Final 32-bit Representation:

Final IEEE 754 Double precision:

Sign	Exponent	Mantissa
1	10000000010	101010000000000000000000.....

Floating-Point Addition:

Add - $13.25 + 2.5$

Step 1 - Convert to binary

$$13.25 = 1.10101 \times 2^3$$

$$2.5 = 1.01 \times 2^1$$

Step 2 - Align the Exponents

The Larger exponent is 3.

convert $1.01 \times 2^1 = 0.0101 \times 2^3$

Now both numbers have the same exponent.

Step 3 - Add the Mantissa.

$$\begin{array}{r} 1.10101 \\ + \\ 0.01010 \\ \hline 1.11111 \end{array}$$

Step 4 - Normalize

The result is already normalized.

$$1.11111 \times 2^3$$

Step 5 - Round

Final Decimal : $13.25 + 2.5 = 15.75$

Precision Loss:

$$0.1 + 0.2 \approx 0.30000000000000004$$

This happens because numbers like 0.1 cannot be represented exactly in binary.

Accuracy of Calculation:

Parameter	Result
Decimal Number	-13.25
Binary Form	1101.01 ₂
Normalized form	1.10101×2^3
Single Precision Sign	1
Single Precision Exponent	10000010
Single Precision Mantissa	1010100000000000....
Double Precision Exponent	10000000010
Addition Example	$13.25 + 2.5 = 15.75$

Interpretation:-

* IEEE 754 stores numbers using sign.

Exponent and Mantissa.

* Single precision uses less memory.

* Double Precision gives more accuracy.

* Some numbers may have small Precision errors.

Code of Conduct:

I K. Nishalini (19252130) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K. Nishalini

MARKS ALLOCATION

	Problem Understanding(20%)	Method Selection(20%)	Solution Process(25%)	Accuracy of Calculation(20%)	Interpretation of Results(15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Clearly interprets problem. identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation